

Documentation

Project Name: HealthTrends – Distributed Health Data Analyzer

ABSTRACT

This project, **HealthTrends – Distributed Health Data Analyzer**, presents an end-to-end distributed data engineering pipeline designed to ingest, store, process, and analyze healthcare data using industry-standard big data technologies. Patient health records are ingested through **Apache Kafka**, validated and stored in a distributed manner using **HDFS** on a **Cloudera QuickStart virtual environment**, and subsequently processed using **Apache Spark (PySpark)** for large-scale batch analytics. The system performs diagnosis-wise trend analysis, computes average patient age, and evaluates gender ratio statistics on a daily basis. A Python-based orchestration layer automates data ingestion, validation, storage, and Spark job execution, ensuring reliability and fault tolerance. The final analytical results are generated as structured CSV reports suitable for downstream analytics and reporting. This project demonstrates a practical and scalable approach to healthcare data processing and highlights essential data engineering concepts such as distributed ingestion, schema validation, batch processing, and workflow orchestration.

1.INTRODUCTION

Healthcare systems generate large volumes of patient data every day. Processing this data efficiently is essential to understand health trends, monitor diseases, and support decision-making.

HealthTrends – Distributed Health Data Analyzer is a data engineering project that implements an **end-to-end distributed batch pipeline** using **Kafka, HDFS, PySpark, and Python orchestration**. The system ingests patient health records, validates and stores them in HDFS, processes them using Spark, and generates daily analytical reports.

1.1 Problem Statement

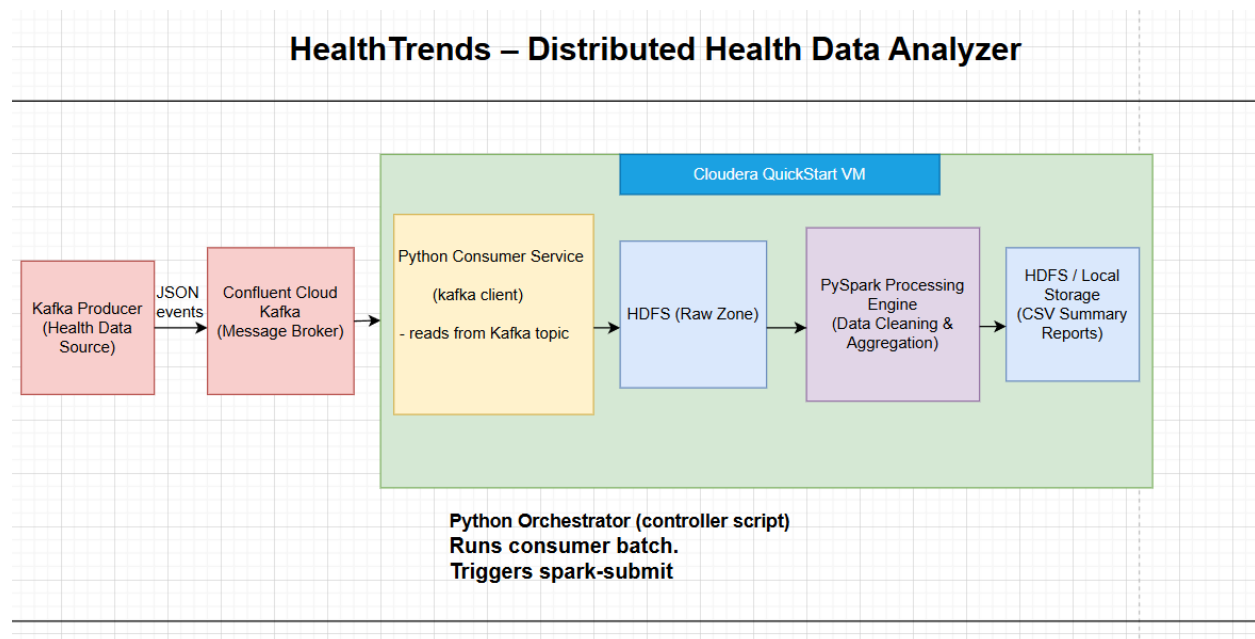
Healthcare organizations generate large volumes of patient data from multiple sources on a daily basis. Processing this data using traditional single-node systems is inefficient and does not scale well with increasing data volume and velocity. There is a need for a distributed data processing framework that can reliably ingest healthcare data, validate its structure, store it in a scalable manner, and perform efficient analytical computations to extract meaningful trends. The absence of an automated, fault-tolerant pipeline for handling healthcare data can lead to delays in analysis, poor data quality, and limited insights. Therefore, a scalable and distributed data engineering solution is required to ingest, process, and analyze healthcare data efficiently while ensuring data integrity and reliability.

1.2. Objectives

The main objectives of this project are:

- To design a distributed data ingestion pipeline using Kafka
- To store healthcare data in a scalable distributed file system (HDFS)
- To process large datasets using PySpark
- To generate daily healthcare trend reports
- To automate the workflow using Python orchestration
- To demonstrate real-world data engineering concepts

2. SYSTEM ARCHITECTURE



Architecture Explanation

1. **Kafka Producer**

Generates and sends patient health records to Kafka topics.

2. **Kafka Broker**

Acts as a distributed message queue to buffer and distribute incoming health events.

3. **Python Kafka Consumer**

- Reads messages from Kafka topics
- Validates data schema
- Filters invalid records
- Prepares data for storage

4. **HDFS (Cloudera QuickStart VM)**

- Stores validated health records
- Uses **date-based partitioning** for efficient access

5. **PySpark Processing Engine**

- Reads JSON data from HDFS
- Performs data cleaning and aggregation
- Computes healthcare trends

6. **CSV Reports**

- Final aggregated results stored back in HDFS
- Ready for analytics and reporting

4. Technology Stack

Component	Technology
Data Ingestion	Apache Kafka
Programming Language	Python
Storage	HDFS
Processing	Apache Spark (PySpark)
Environment	Cloudera QuickStart VM
Orchestration	Python Script

5. Data Pipeline Workflow

Step 1: Kafka Data Ingestion

- Patient health events are published by Kafka Producers.
- Kafka ensures fault-tolerant and scalable ingestion.

Step 2: Data Validation and Storage

- Python Consumer reads Kafka messages.
- Schema validation is applied:
 - Missing fields are rejected
 - Invalid gender/age values are logged
- Valid records are converted to JSON.
- Data is stored in HDFS using date partitions:

/healthTrend/patient_json/date=YYYY-MM-DD/

Step 3: Distributed Data Processing (PySpark)

- PySpark reads JSON data from HDFS.
- Data transformations include:
 - Type casting
 - Filtering invalid records
- Aggregations performed:
 - Diagnosis-wise patient count
 - Average patient age
 - Gender ratio calculation

Step 4: Report Generation

- Final aggregated data is written to HDFS in CSV format:

/healthTrend/reports/trend_summary/date=YYYY-MM-DD/

6. Orchestration Logic

A Python orchestration script controls the entire pipeline:

- Triggers Kafka ingestion
- Handles HDFS directory creation and cleanup
- Submits Spark jobs using spark-submit
- Performs HDFS safemode checks
- Implements retry logic and logging
- Ensures pipeline completion for each date

7. How to Run the Project

Prerequisites

- Cloudera QuickStart VM
- Python installed
- HDFS and Spark services running

Execution Command

```
python patient_data_pipeline.py YYYY-MM-DD/
```

8.HDFS Output

```
diagnosis,count,avg_age,gender_ratio  
E11,2986,47.83,0.4973  
J45,2938,47.63,0.5044  
I25,2917,47.78,0.505  
I10,2895,47.5,0.5105
```

9. Repository Structure

healthtrend-pipeline/

- |— kaka/ # Kafka producer & consumer scripts
- |— spark/ # PySpark processing job
- |— orchestrator/ # Python orchestration script
- |— docs/ # PRD, HLD, LLD documents
- |— architecture.png # System architecture diagram
- |— README.md
- |— PROJECT_DOCUMENTATION.md

10. Key Features

- Distributed data ingestion using Kafka
- Schema validation and error handling
- HDFS-based scalable storage
- PySpark batch analytics
- Date-partitioned reporting
- Fully automated orchestration
- Production-style logging and retries

11. Use Cases

- Healthcare trend analysis
- Disease monitoring
- Population health analytics
- Batch data engineering pipelines
- Educational and internship demonstrations

12. Conclusion

- This project successfully demonstrates a **real-world distributed healthcare data processing pipeline** using industry-standard big data technologies. It highlights practical data engineering skills including ingestion, validation, distributed storage, batch processing, and workflow orchestration.